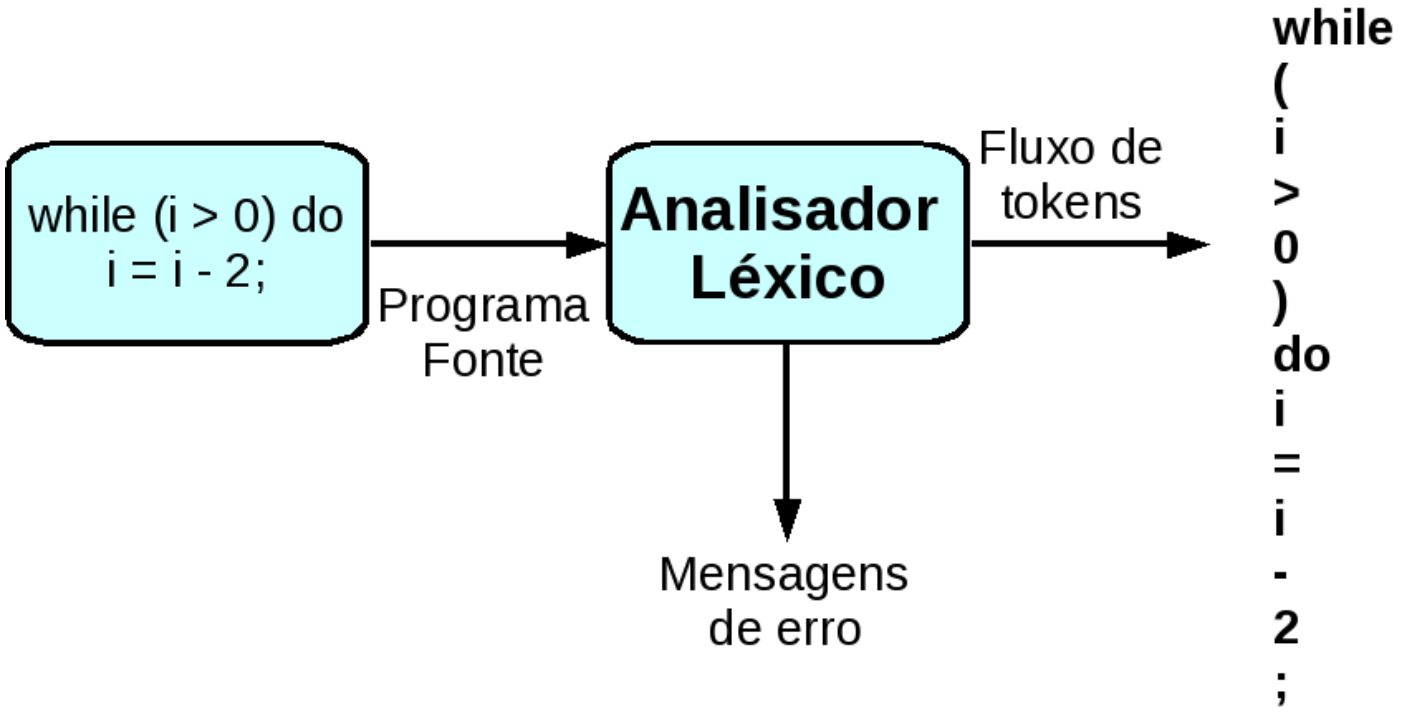
Análise Léxica

Prof^{a}. Thatyana de Faria Piola Seraphim

thatyana@unifei.edu.br

Análise Léxica

A principal tarefa do analisador léxico é ler os caracteres de entrada do programa fonte, agrupá-los em lexemas e produzir como saída uma sequência de *tokens*.



Análise Léxica

Outras tarefas que o analisador léxico pode realizar:

- Remover comentários, espaços em branco, quebra de linha, tabulação e outros caracteres que são usados para separar os *tokens* de entrada.
- Correlacionar as mensagens de erro geradas pelo compilador com o programa fonte.
- O analisador léxico faz uma cópia do programa fonte com as mensagens de erro inseridas nas posições apropriadas.

Os analisadores léxicos são divididos em dois processos:

- **Escandimento ou Varredura**: processo de simples varredura da entrada sem se preocupar com a remoção de comentários e a compactação de caracteres de espaço em branco em apenas um caractere.
- **Análise léxica**: é a parte mais complexa, onde o analisador léxico produz uma sequência de *tokens* como saída.

Análise Léxica

Tokens, Padrões e Lexemas

Na análise léxica, três termos estão relacionados:

- **Token**: é um par que consiste de um nome e um atributo.
  - O nome do *token* é um símbolo que representa um tipo de unidade lógica (palavra-chave ou uma sequência de caracteres de entrada denotando um identificador).
  - Os nomes dos *tokens* são os símbolos de entrada que o analisador sintático processa.
- **Padrão**: é uma descrição da forma que os lexemas de um *token* podem assumir.
  - **Palavra-chave**: o padrão é apenas uma sequência de caracteres que formam a palavra chave.
  - **Identificadores**: o padrão é uma estrutura mais complexa, que é casada por muitas sequências de caracteres.

- **Lexema**: é uma sequência de caracteres no programa fonte que casa com o padrão para um *token* e é identificado pelo analisador léxico como uma instância deste *token*.

Token	Descrição Formal	Exemplos de Lexemas
if	caracteres i, f	`if`
else	caracteres e, l, s, e	`else`
comparacao	$<, >, <=, >=, ==, !=$	`<=, !=`
id	letra seguida por letras e dígitos	`pi`
num	qualquer constante numérica	`3.1416, 0`
literal	qualquer caractere diferente de "	`"unifei"`

Algumas classes que abrangem a maioria ou todos os *tokens*:

1. Um *token* para cada palavra chave.
2. *Tokens* para os operadores, seja individualmente ou em classes (*token* `comparacao`).
3. *Token* representando todos os identificadores.
4. Um ou mais *tokens* representando constantes.
5. *Tokens* para cada símbolo de pontuação (parênteses, vírgula, ponto-e-vírgula).

Análise Léxica

Atributos de Tokens

Quando mais de um lexema casar com um padrão:

- O analisador léxico precisa oferecer às próximas fases, informações adicionais sobre qual foi o lexema casado.
  - Por exemplo: o *token* **num** casa com 0 e 1, mas é importante para o gerador de código saber qual lexema foi encontrado no programa.
- O analisador léxico retorna ao analisador sintático não apenas um nome de *token*.
  - Um valor de atributo que descreve o lexema representado pelo *token*.
- O nome do *token* influencia as decisões durante a análise sintática, enquanto que o valor do atributo influencia a tradução dos *tokens* após o reconhecimento sintático.

Expressões regulares:

- são uma importante notação para especificar os padrões de lexemas.
- Nem todos os padrões possíveis podem ser expressos. As expressões regulares são muito eficientes na especificação dos tipos de padrões de que realmente é preciso para os *tokens*.

Especificação de *Tokens*

Expressões Regulares

- **Alfabeto (Σ)**: é um conjunto de símbolos finito e não vazio. Os alfabetos comuns incluem:
  1. Alfabeto binário: $\Sigma = \{0, 1\}$.
  2. Conjunto de todas as letras minúsculas: $\Sigma = \{a, b, \ldots, z\}$.
  3. Conjunto de todos os caracteres ASCII.
- **String (cadeia)**: uma cadeia finita de símbolos escolhidos de algum alfabeto. Exemplo: 01101 (é uma *string* do alfabeto binário).
  - **String vazia (ε)**: com zero ocorrências de símbolos, ou seja, é uma *string* que pode ser escolhida de qualquer alfabeto.
  - **Comprimento da string** $|w|$: o número de posições para símbolos. Por exemplo: 01101 tem comprimento 5, ou $|011| = 3$ e $|\varepsilon| = 0$.
  - O conjunto de todas as *strings* sobre um alfabeto Σ é denotado por Σ^*.

Especificação de *Tokens*

Expressões Regulares

- **Linguagem**: é um conjunto de *strings*, todas escolhidas a partir de algum Σ^*, onde Σ é um alfabeto específico.
- Se Σ é um alfabeto, e $L \subseteq \Sigma^*$, então L é uma *linguagem sobre* Σ.
- Exemplos:
  1. Linguagem de todas as *strings* que consistem de n O's seguidos por n valores de 1, para $n \geq 0$: $\{\varepsilon, 01, 0011, 000111, \ldots\}$.
  2. Conjunto de *strings* de 0's e 1's com um número igual de cada um deles: $\{\varepsilon, 01, 10, 0011, 0101, 1001, \ldots\}$.
  3. Conjunto de números binários cujo valor é um número primo: $\{10, 11, 101, 111, 1011, \ldots\}$.
  4. Σ^* é uma linguagem para qualquer alfabeto Σ.
  5. $\emptyset$, a linguagem vazia, é uma linguagem sobre qualquer alfabeto. Não possui nenhuma *string*.
  6. $\{\varepsilon\}$, a linguagem consiste apenas na *string* vazia. É uma linguagem sobre qualquer alfabeto.

Expressões regulares:

- São outro tipo de notação para a definição de linguagens.
- Podem ser consideradas *linguagens de programação*, na qual pode-se expressar algumas aplicações importantes.
- Estão relacionadas aos autômatos finitos e podem ser consideradas uma alternativa amigável para o usuário descrever *software*.
- Oferecem um modo declarativo de expressar *strings* que serão aceitas.

Especificação de *Tokens*

Expressões Regulares

As expressões regulares servem como a linguagem de entrada para muitos sistemas que processam *strings*, como:

- Comandos de pesquisa como o **grep** utilizado para localizar *strings*. Em geral, esses sistemas usam uma notação semelhante à de expressões regulares para descrever padrões que o usuário quer encontrar em um arquivo.
- Geradores de analisadores léxicos (Flex e Lex) que aceitam descrições das formas de *tokens* que são expressões regulares.

Especificação de *Tokens*

Expressões Regulares

A expressão regular $01^* + 10^*$ denota a linguagem que consiste em todas as *strings* que são:

- um único 0 seguido por qualquer número de 1's;
- um único 1 seguido por qualquer número de 0's.

Existem três operações que os operadores de expressões regulares representam:

- União.
- Concatenação.
- Fechamento.

Expressões Regulares

Operações

- **União (|)** união de duas linguagens L e M, denotada por $L \cup M$, é o conjunto de *strings* que estão em L ou M, ou em ambas.
  Por exemplo: se $L = \{001, 10, 111\}$ e $M = \{\varepsilon, 001\}$, então $L \cup M = \{\varepsilon, 10, 001, 111\}$
- **Concatenação**: a concatenação de linguagens L e M é o conjunto de *strings* que podem ser formadas tomando-se qualquer *string* em L e concatenando-se essa *string* com qualquer *string* em M. A concatenação de linguagens é denotada por um ponto (.) ou sem nenhum operador.
  Por exemplo: $L = \{001, 10, 111\}$ e $M = \{\varepsilon, 001\}$, então $L.M$ é $LM = \{001, 10, 111, 001001, 10001, 111001\}$.

- **Fechamento ou estrela**: o fechamento de uma linguagem L é denotado por L^* e representa o conjunto das *strings* que podem ser formadas tomando-se qualquer número de *strings* de L, possivelmente com repetições e concatenação.
  Por exemplo:
  - Se $L = \{0, 1\}$, então L^* consiste nas *strings* de 0's e 1's.
  - Se $L = \{0, 11\}$, então L^* consiste nos pares de 0's e 1's tais que os símbolos 1 formam pares. Por exemplo: 011, 11110 e ε.

Expressões Regulares

Definição Formal

Dado que L é uma **expressão regular** se L for:

1. a para algum a no alfabeto Σ.
2. ε.
   Nos itens 1 e 2, as expressões regulares representam as linguagens $\{a\}$ e $\{\varepsilon\}$.
3. $\emptyset$ (representa a linguagem vazia).
4. $(L_1 \cup L_2)$, onde L_1 e L_2 são expressões regulares.
5. $(L_1 \circ L_2)$, onde L_1 e L_2 são expressões regulares.
6. (L_1^*), onde L_1 é uma expressão regular.
   Nos itens 4, 5 e 6, as expressões representam as linguagens tomando-se a união e a concatenação de L_1 e $L2$ ou a estrela da linguagem L_1.

Expressões Regulares

Definição Formal – Exemplo

A expressão regular (a)|(b)*(c), pode ser substituída por $\mathbf{a}|\mathbf{b}^*\mathbf{c}$:

- As duas expressões representam o conjunto de cadeias que são um único *a* ou são zero ou mais *b* seguidos por um *c*.
- Por exemplo: considere $\Sigma = \{a, b\}$, a expressão regular $\mathbf{a}|\mathbf{b}$ representa a linguagem {a,b}.
  1. $(\mathbf{a}|\mathbf{b})(\mathbf{a}|\mathbf{b})$, representa $\{aa,ab,ba,bb\}$, a linguagem de todas as cadeias de tamanho dois sob o alfabeto Σ. Outra expressão regular para a mesma linguagem é aa|ab|ba|bb.
  2. $\mathbf{a}^*$ consiste em todas as cadeias de zero ou mais *as* $\{\varepsilon, a, aa, \ldots\}$.
  3. $(\mathbf{a}|\mathbf{b})^*$: todas as cadeias com zero ou mais *a* ou *b* $\{\varepsilon, a, b, aa, ab, ba, bb, aaa, \ldots\}$, ou seja, todas as cadeias de *as* e *bs*. Outra expressão regular para a mesma linguagem é $(a^*b^*)^*$.
  4. $\mathbf{a}|\mathbf{a}^*\mathbf{b}$: representa a linguagem $\{a,b,ab,aab,aaab,\ldots\}$, consistindo da cadeia *a* e todas as cadeias consistindo em zero ou mais *as* terminado em *b*.

Expressões Regulares

Precedência de Operadores

- Os operadores nas expressões regulares têm uma ordem de precedência pré-definida, ou seja, os operadores estão associados com seus operandos em uma ordem específica:
  - **Operador unário (*)**: é o de precedência mais alta, ou seja, ele se aplica apenas à menor sequência de símbolos à sua esquerda que seja uma expressão regular bem formada.
  - **Operador de concatenação (.)**: todas as expressões que são justapostas são agrupadas juntas. É um operador associativo, não importa em que ordem é agrupado as concatenações, se existir uma opção, deve-se agrupar a partir da esquerda.
  - **Operador união (+)**: são agrupadas com seus operandos. Uma vez que a união é uma operação associativa, a ordem que são agrupadas as uniões é pouco importante, mas deve-se partir da esquerda.

Expressões Regulares

Definições Regulares

Pode-se dar nomes a certas expressões regulares:

- Os nomes podem ser usados em expressões subsequentes como se fossem os próprios símbolos.
- Os identificadores em C são cadeias de letras ou dígitos. A definição regular para os identificadores da linguagem C, podem ser:

Identificadores

$letra \rightarrow \texttt{A}|\texttt{B}|\ldots|\texttt{Z}|\texttt{a}|\texttt{b}|\ldots|\texttt{z}$
$digito \rightarrow \texttt{0}|\texttt{1}|\ldots|\texttt{9}$
$ident \rightarrow letra(letra|digito)^*$

- Números sem sinal (inteiros ou ponto flutuante), são cadeias como 5280, 0.012, 6.33E4 ou 1.89E-4. Como ficaria a definição regular para estes números?

- Números sem sinal (inteiros ou ponto flutuante), são cadeias como 5280, 0.012, 6.33E4 ou 1.89E-4. Como ficaria a definição regular para estes números?

Definição regular

$digito \rightarrow \texttt{0} \mid \texttt{1} \mid \ldots \mid \texttt{9}$
$digitos \rightarrow digito\ digito^*$
$numFrac \rightarrow .digitos \mid \varepsilon$
$numExp \rightarrow (\mathrm{E}(+ \mid \text{-} \mid \varepsilon)digitos) \mid \varepsilon$
$num \rightarrow digitos\ numFrac\ numExp$

Expressões Regulares

Extensões de Expressões Regulares

Algumas extensões que são importantes na especificação de analisadores léxicos:

- **Uma ou mais instâncias (+)**: representa o fechamento positivo de uma expressão regular e sua linguagem. Deve existir pelo menos uma instância.
- **Zero ou uma instância (?)**: o operador ? significa zero ou uma ocorrência.
- **Classes de caracteres**: quando $a_1, a_2, \ldots, a_n$ formam uma sequência lógica (por exemplo, letras maiúsculas consecutivas, letras minúsculas ou dígitos), a sequência pode ser substituída por a_1-a_n, ou seja, apenas a primeira e a última separadas por um hífen.
  Por exemplo: [**abc**] é a abreviação de **a**|**b**|**c**, e [a–z] é a abreviação de **a**|**b**| ... |**z**.

Expressões Regulares

Atividade

Com o uso das abreviações, é possível reescrever a definição regular:

Definição regular

$\mathit{letra} \rightarrow \mathtt{A}|\mathtt{B}|\ldots|\mathtt{Z}|\mathtt{a}|\mathtt{b}|\ldots|\mathtt{z}$
$\mathit{digito} \rightarrow \mathtt{0}|\mathtt{1}|\ldots|\mathtt{9}$
$\mathit{ident} \rightarrow \mathit{letra}(\mathit{letra}|\mathit{digito})^*$

Expressões Regulares

Atividade

Com o uso das abreviações, é possível reescrever a definição regular:

Definição regular

$letra \rightarrow \texttt{A}|\texttt{B}|\ldots|\texttt{Z}|\texttt{a}|\texttt{b}|\ldots|\texttt{z}$
$digito \rightarrow \texttt{0}|\texttt{1}|\ldots|\texttt{9}$
$ident \rightarrow letra(letra|digito)^*$

Reescrevendo a Definição Regular

$letra \rightarrow$ [A–Za–z]
$digito \rightarrow$ [0–9]
$ident \rightarrow letra(letra|digito)^*$

Expressões Regulares

Atividade

Definição regular

$digito \rightarrow 0 \mid 1 \mid \ldots \mid 9$
$digitos \rightarrow digito\ digito^*$
$numFrac \rightarrow .digitos \mid \varepsilon$
$numExp \rightarrow (\mathrm{E}(+ \mid - \mid \varepsilon)digitos) \mid \varepsilon$
$num \rightarrow digitos\ numFrac\ numExp$

Expressões Regulares

Atividade

Definição regular

$digito \rightarrow \texttt{0} \mid \texttt{1} \mid \ldots \mid \texttt{9}$
$digitos \rightarrow digito\ digito^*$
$numFrac \rightarrow .digitos \mid \varepsilon$
$numExp \rightarrow (\mathrm{E}(+ \mid \text{-} \mid \varepsilon)digitos) \mid \varepsilon$
$num \rightarrow digitos\ numFrac\ numExp$

Reescrevendo a Definição Regular

$digito \rightarrow [0\text{–}9]$
$digitos \rightarrow digito^+$
$num \rightarrow digitos(.digitos)?(\mathrm{E}[+\ \text{-}]?digitos)?$